

An Introduction to Universal Artificial Intelligence

Chapter 11: AIXI-MDP

Based on Hutter, Quarel & Catt

CRC Press, 2024

"Simplicity is the ultimate sophistication." — Leonardo da Vinci

Outline

- 1 11.1 AIXI-MDP Setup
- 2 11.2 Definition of AIXI-MDP
- 3 11.3 Experimental Results
- 4 11.4 Exercises
- 5 11.5 History and References

From AIXI to a Computable Approximation I

Where we are. Earlier chapters introduced universal (Bayesian) models of prediction (sequence prediction, CTW) and decision-making (the general history-based agent $AI\xi$ (ξ is pronounced “xi”), and its universal instantiation AIXI). AIXI served as a *gold standard*: a mathematically well-defined, maximally intelligent agent that we can use to judge other agents, even though AIXI itself is not something we can run on a computer.

Why AIXI cannot be run directly. Two obstacles stand in the way:

- The Kolmogorov complexity $K(x)$ (the length of the shortest program that outputs the string x – a measure of how “simple” or “compressible” x is) is **incomputable**: no algorithm can calculate it in general. Since the Solomonoff mixture $M = \xi_U$ is built out of $K(x)$, it is incomputable too.
- Even if we could evaluate the mixture, the **expectimax** search (alternating maximization over actions and expectation over observations, all the way to the planning horizon) that AIXI performs is computationally intractable for essentially any real problem: its cost explodes exponentially with the horizon.

From AIXI to a Computable Approximation II

$K(x)$ is Kolmogorov complexity: the length, in bits, of the shortest computer program that prints the string x and then stops. It is a rigorous notion of “simplicity,” but famously cannot be computed by any algorithm. $M = \xi_U$ is the Solomonoff mixture: a weighted average over all computable environments, weighting simpler (lower- K) environments more heavily – it is the ideal universal predictor, but inherits $K(x)$'s incomputability.

From AIXI to a Computable Approximation III

The plan for this part of the book. Since we cannot use AIXI as-is, we build *systematic computable approximations* of it:

- **This chapter (11):** the first and simplest instantiation of $\text{AI}\xi$, called **AIXI-MDP**, where ξ is a very efficiently computable mixture over **Markov Decision Processes (MDPs)**. An MDP is a simple kind of environment whose next observation depends only on the most recent action and observation, not the full history.
- **Chapter 12:** a more advanced, more broadly applicable instantiation of $\text{AI}\xi$.
- **Chapter 13:** some further approximations that are powerful in theory but not yet practical.

This chapter's testbed. We run AIXI-MDP on the classic repeated 2×2 matrix games from Section 10.2: *Prisoner's Dilemma*, *Stag Hunt*, *Chicken*, *Battle of the Sexes*, and *Matching Pennies*. Because the action and observation spaces here are just $\{0, 1\}$ (one bit each), the action/perception spaces are small enough that we can **brute-force compute** the Bayes-optimal policy π_ξ^* directly from the expectimax expression (Theorem 7.4.2), with no further approximation needed beyond restricting ξ to MDPs.

Emulating Simultaneous Actions I

Repeated matrix games are usually *simultaneous*: both players choose their action at the same time, each ignorant of what the other just chose. AIXI, however, was described using the usual **cybernetic model** for reinforcement learning, in which there is a strict order: the agent acts, the environment receives that action, and only afterwards does the environment send back a *percept* (an observation-reward pair). These two pictures seem incompatible – but we can reconcile them with a small trick.

The trick. We hide the opponent *inside* the environment, and we constrain the type of distribution the opponent (as part of the environment) is allowed to use: the opponent's policy is not allowed to depend on the agent's most recently issued action. This exactly captures “the opponent did not see what I just did.”

Emulating Simultaneous Actions II

One iteration of the interaction loop:

- The agent takes action a_t , but the environment does not yet receive it.
- The opponent (who is part of the environment) takes action a'_t , ignorant of the agent's choice of a_t .
- The environment receives *both* actions a_t and a'_t , and then, using the **reward matrix** $\mathfrak{R}$ ($\mathfrak{R}$ is a capital Fraktur R – the pay-off matrix from game theory, Section 10.2.2), computes the reward r_t for the agent (taking a_t against opponent action a'_t) and the reward r'_t for the opponent (taking a'_t against agent action a_t).
- The agent receives reward r_t and observation $o_t = a'_t$. The opponent receives reward r'_t and observation $o'_t = a_t$. In other words: each player is told the reward matrix entry they earned, plus, as extra side information, the other player's most recently played action.
- The time step increments, $t := t + 1$, and we loop.

Emulating Simultaneous Actions III

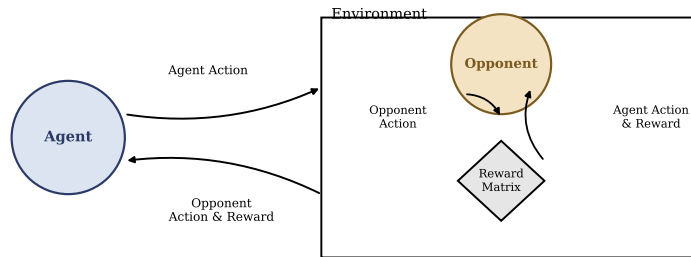


Figure 11.1. How repeated normal-form games with simultaneous actions are realized within the usual cybernetic (act-then- perceive) model, by hiding the opponent inside the environment and withholding the most recent action from it.

Emulating Simultaneous Actions IV

Environment-agent (a)symmetry. The environment is *not* the same thing as the opponent: the environment also includes the reward matrix and manages the whole interaction loop. From the agent's point of view, the opponent is just one part of a larger environment. Symmetrically, we could redraw Figure 11.1 from the opponent's perspective, in which case the opponent would view the reward matrix, the interaction loop, *and* the agent together as its own “environment.”

Implicit knowledge. Both players are implicitly aware of the other's most recently received reward as well (r'_{t-1} for the agent, r_{t-1} for the opponent), since both players know the full reward matrix $\mathfrak{R}$ and each other's last action. This lets us simplify notation: we can write the agent's observation as $o_t = a'_t$ (just the opponent's action) rather than the pair $o_t = (a'_t, r'_t)$, without losing any information, because r'_t is fully determined by (a_t, a'_t) and $\mathfrak{R}$.

Known Reward Matrix and a Markov Opponent I

Withholding the last action from the history. Since in this chapter the agent is told the reward matrix $\mathfrak{R}$ in advance, the reward $r_t = \mathfrak{R}(a_t, o_t)$ is just a known deterministic function of the action a_t and observation o_t – so we can safely drop r_t from the history and only track the *observation-action* history

$$h_{<t} := (oa)_{<t} = o_1 a_1 \dots o_{t-1} a_{t-1}.$$

$h_{<t}$ (read: “*h sub less-than t*”) denotes everything the agent has seen and done before time t . Notice the order: observation then action within each round, because o_t depends on the previous observation and action, but never on the agent’s current action a_t – this is exactly the “opponent can’t see what I just did” property we built into the loop above.

A footnote in the book flags that Chapter 12 will relax this: there, the agent is *not* told the reward matrix or even the rules of the game in advance, and must learn those too.

Known Reward Matrix and a Markov Opponent II

Markov environments. The agent additionally *assumes* that the environment (and hence the opponent inside it) is a **Markov Decision Process (MDP)**: the next observation only depends on the single most recent action-observation pair, not on the entire history. Formally, if μ (μ is pronounced “mu” and denotes the true environment) is Markov, then

$$\mu(o_t \mid h_{<t}) = \mu(o_t \mid o_{t-1}a_{t-1}).$$

This says: to predict the next observation o_t , you only need to remember the very last observation-action pair (o_{t-1}, a_{t-1}) – everything further back in the history is irrelevant. This is what “Markov” means: no long-term memory is needed.

Even though the *environment* is assumed Markov, the *agent itself* remains history-based: it still conditions its action choice on the entire interaction history $h_{<t}$, hoping to detect and exploit statistical regularities in how the opponent responds.

Known Reward Matrix and a Markov Opponent III

No Grain of Truth. These two assumptions – known reward matrix, Markov opponent – can cause trouble. Consider AIXI-MDP playing against a copy of itself. Neither agent can learn that its opponent is *also* another instance of AIXI-MDP; instead, each concentrates its posterior belief ξ on whichever model $\nu \in \mathcal{M}$ (ν is pronounced “nu” and denotes a candidate environment model; $\mathcal{M}$ is the model class, i.e., the whole collection of models the agent entertains) best fits the observations so far. In other words, the class $\mathcal{M}_{\text{MDP}}$ of Markov models does **not** contain a *Grain of Truth* (recall Section 10.6: a model class “has a Grain of Truth” if the true environment, or something behaviourally equivalent to it, is actually inside the class).

This is a deep, general problem, not just an MDP quirk: even playing a true copy of AIXI against itself, using the richer model class $\mathcal{M}_{\text{sol}}$ of *all* computable environments, runs into the same issue – AIXI is not contained in its own model class, because AIXI itself is non-computable! The only known model class with a genuine Grain of Truth is the class of **Reflective Oracle** machines $\mathcal{M}_r^O \supset \mathcal{M}_{\text{comp}}$ (Theorem 10.6.2), which we will not need for AIXI-MDP.

Binary Action and Observation Space

This whole setup is general – it works for any class of games with any number of actions or opponents. In this chapter we specialize to the simplest interesting case:

The 2×2 matrix-game setting of this chapter

- A single opponent, and repeated 2×2 matrix games.
- Both the action space and the observation space are binary: $\mathcal{A} = \mathcal{O} = \{0, 1\}$ ($\mathcal{A}, \mathcal{O}$ are the sets of possible actions/observations – “script A,” “script O”).
- The observation o_t is exactly the action the opponent chose on the previous round.
- In each game the agent and opponent are given a reward matrix $\mathfrak{R}$, giving the reward for each observation-action pair (o, a) .
- The agent additionally has access to the full history $h_{<t}$, which records every action taken and every observation received by both players so far (implicitly – see the previous frame).

A “ 2×2 matrix game” just means: there are 2 players, each with 2 possible actions (say, “cooperate”/“defect” or “swerve”/“go straight”), and the reward for each combination of the two players’ choices is listed in a 2×2 table – the reward matrix $\mathfrak{R}$. This is exactly the classical setting used for games like Prisoner’s Dilemma, discussed in Section 10.2.3 and revisited in Section 11.3 below.

Python: One Round of the Interaction Loop

A concrete, tiny example of the loop from Figure 11.1, using the classic Prisoner's Dilemma reward matrix (action/observation 0 = "cooperate", 1 = "defect"): mutual cooperation pays 3, mutual defection pays 1, and defecting against a cooperator pays 5 (with the sucker getting 0.)

```
1 # Reward matrix R[a][o]: reward to the agent for action a
2 # against opponent action (=observation) o.
3 R = {(0, 0): 3, (0, 1): 0,      # agent cooperates: 3 if opp. cooperates, 0 if opp. defects
4       (1, 0): 5, (1, 1): 1}    # agent defects:      5 if opp. cooperates, 1 if opp. defects
5
6 def one_round(a_t, a_opp):
7     """One iteration of Figure 11.1's loop (rewards computed simultaneously)."""
8     r_t      = R[(a_t, a_opp)]    # agent's reward
9     r_opp    = R[(a_opp, a_t)]    # opponent's reward (R read with roles swapped)
10    o_t, o_opp = a_opp, a_t        # each side observes the other's action
11    return r_t, o_t, r_opp, o_opp
12
13 # Agent defects (1), opponent (e.g. "tit-for-tat") cooperates (0) this round:
14 r_t, o_t, r_opp, o_opp = one_round(a_t=1, a_opp=0)
15 print(f"agent reward={r_t}, agent obs={o_t} | opp reward={r_opp}, opp obs={o_opp}")
16 # -> agent reward=5, agent obs=0 | opp reward=0, opp obs=1
```

The agent defects while the opponent cooperates, so the agent gets the "temptation" reward of 5 and observes that the opponent cooperated ($o_t = 0$); the opponent gets the "sucker" reward of 0 and observes that the agent defected ($o_{opp} = 1$).

Recursive Q-Value Function of AIXI-MDP I

Definition. AIXI-MDP is simply $\text{AI}\xi$ (recall: the general Bayesian agent) with the model class $\mathcal{M}$ set to $\mathcal{M}_{\text{MDP}}$, the class of Markov Decision Processes over $\mathcal{A} \times \mathcal{O}$, described below. Any of the three equivalent characterizations of $\text{AI}\nu$ from Chapter 6 can be adapted here: the abstract expectation-policy forms (Definitions 6.6.1, 6.7.1), the explicit iterative form (Lemma 6.6.3), or the (pseudo)recursive Bellman form (Theorem 6.7.2). We use the last one, with $\nu = \xi$.

In strategic game theory it is common to *neither discount nor normalize* rewards. Plugging this choice ($\gamma_t = 1 = \Gamma_t$; γ is “gamma,” the discount factor, and Γ_t its running normalizer) into the general Bellman optimality equation (6.7.3)/(6.7.5) for the optimal policy π_ξ^* gives

$$Q_\xi^{*,m}(h_{<t}, a_t) = \sum_{e_t} \xi(e_t \mid h_{<t}a_t) \left[r_t + \max_{a_{t+1}} Q_\xi^{*,m}(h_{1:t}, a_{t+1}) \right]$$

with history $h_{1:t} := a_1 o_1 r_1 \dots a_t o_t r_t$.

$Q_\xi^{*,m}(h_{<t}, a_t)$ is the optimal Q-value: the best total reward the agent can expect to collect from now (time t) up to the planning horizon m , if it takes action a_t right now and then plays optimally afterwards, given everything it has seen ($h_{<t}$) and believing the mixture ξ . The sum is over $e_t = (o_t, r_t)$, all possible percepts (observation+reward pairs) the environment could send back; $\xi(e_t \mid h_{<t}a_t)$ is how likely ξ thinks each one is.

Recursive Q-Value Function of AIXI-MDP II

Simplifying for known, deterministic rewards. Now $r_t = \mathfrak{R}(a_t, o_t)$ is a known deterministic function of a_t and o_t , so we can drop r_t from the history entirely, and the percept $e_t = o_t r_t$ becomes just o_t . Moreover, since the environment (the opponent) never sees the agent's current action a_t before choosing its own action $a'_t = o_t$, we only need to consider models $\nu \in \mathcal{M}$ for which $\nu(e_t \mid h_{<t} a_t)$ – and hence $\xi(e_t \mid h_{<t} a_t)$ – does not actually depend on a_t . Putting this together:

Equation (11.2.1) — AIXI-MDP's recursive Q-value

$$Q_{\xi}^{*,m}(h_{<t}, a_t) = \sum_{o_t} \xi(o_t \mid h_{<t}) \left[\mathfrak{R}(a_t, o_t) + \max_{a_{t+1}} Q_{\xi}^{*,m}(h_{1:t}, a_{t+1}) \right]$$

Plain English: to value taking action a_t now, weigh each possible opponent response o_t by how likely ξ thinks it is, add the immediate reward $\mathfrak{R}(a_t, o_t)$ that response would earn, plus the best value achievable from then on (the $\max_{a_{t+1}}$ term, computed the same way one step deeper) – and sum it all up. The recursion **terminates** once t exceeds the horizon m , where we set $Q = 0$ (there is no more future to plan for).

The MDP Model Class and the Mixture ξ_{MDP} I

Computing ξ_{MDP} . The agent assumes the environment is Markov and cannot see its own last action, i.e. $\mu(e_t \mid h_{<t}a_t) = \mu(o_t \mid o_{t-1}a_{t-1})$. $\mathcal{M}_{\text{MDP}}$ is the class of *all* environments ν satisfying this property, and ξ_{MDP} is a **Bayesian mixture** over $\mathcal{M}_{\text{MDP}}$ with a prior w specified below.

A Bayesian mixture is a weighted blend of many candidate models, where models that fit the data seen so far get more weight over time. Here the candidate models are all possible Markov transition rules over the binary action/observation space, and ξ_{MDP} is the specific blended predictor built from them.

Every Markov environment over $\mathcal{A} = \mathcal{O} = \{0, 1\}$ can be encoded as a vector $(\theta_{00}, \theta_{01}, \theta_{10}, \theta_{11}) \in [0, 1]^4$ (θ is pronounced “theta”), one number θ_{ao} for each of the 4 possible most-recent action-observation pairs $(o_{t-1}, a_{t-1}) \in \{(0, 0), (0, 1), (1, 0), (1, 1)\}$, giving the probability that the opponent produces observation $o_t = 0$ given that pair.

The MDP Model Class and the Mixture ξ_{MDP} II

The prior. For the mixture, we assume each θ_{ao} is independently and uniformly distributed, $\theta_{ao} \sim \mathcal{U}([0, 1])$, i.e. following the *principle of indifference* (all values in $[0, 1]$ equally likely a priori): prior density $w(\theta) = 1$ for $\theta \in [0, 1]^4$.

Counting statistics. Let n_{ao}^t be the number of times the action-observation pair ao has appeared in $h_{<t}$, and let $n_{ao \cdot o'}^t$ be the number of those times that were *followed* by the next observation o' . The dot “.” just means “we don’t care what action came next,” since the environment is independent of it. Formally,

$$n_{ao}^t := |\{\tau < t : a_\tau = a \wedge o_\tau = o\}|, \quad n_{ao \cdot o'}^t := |\{\tau < t : a_\tau = a \wedge o_\tau = o \wedge o_{\tau+1} = o'\}|.$$

τ (pronounced “tau”) is just a dummy index running over past time steps. n_{ao}^t counts how many times, somewhere in the history so far, the pair (action a , observation o) occurred and was itself followed by another recorded observation; $n_{ao \cdot o'}^t$ additionally requires that following observation to equal o' . These are exactly the “successes” and “trials” we need for a Bernoulli (coin-flip-like) estimate of θ_{ao} .

The MDP Model Class and the Mixture ξ_{MDP} III

This 1st-order Markov process was already studied in Section 4.2.1. The sub-process of just the o_{t+1} 's that follow $a_t o_t = ao$ is a **Bernoulli**(θ_{ao}) **process** (θ_{ao} being the unknown success probability).

Bayes-mixing it with a uniform prior over θ_{ao} gives the familiar **Laplace estimator**, derived earlier in Section 2.4.3. In our notation:

Equation (11.2.2) — The MDP estimator ξ_{MDP} (Laplace rule)

$$\xi_{\text{MDP}}(o_t \mid h_{<t}) = \frac{n_{a_{t-1}o_{t-1} \cdot o_t}^t + 1}{n_{a_{t-1}o_{t-1}}^t + 2}$$

To predict the next observation o_t , look only at the most recent action-observation pair (a_{t-1}, o_{t-1}) (Markov assumption), count how many times in the past that exact pair was followed by o_t (numerator, plus a pseudo-count of 1), and divide by how many times that pair occurred at all (denominator, plus a pseudo-count of 2 – one for each possible outcome $o_t \in \{0, 1\}$). This “add-one” correction is exactly Laplace’s rule of succession: with zero data it defaults sensibly to probability $\frac{1}{2}$, and it never outputs exactly 0 or 1.

The MDP Model Class and the Mixture ξ_{MDP} IV

Note $\xi_{\text{MDP}} \notin \mathcal{M}_{\text{MDP}}$: unlike any single $\nu \in \mathcal{M}_{\text{MDP}}$, ξ_{MDP} itself depends on the *whole* history (through the counts), not just the last pair – computing ξ_{MDP} has been reduced to counting occurrences of a substring in a string, which is cheap: $O(1)$ time per t , done incrementally.

Comparison to the KT estimator (Definition 4.1.1): the KT estimator uses a slightly different (Jeffreys) prior (Definition 2.4.11), which *halves* the 1 and 2 appearing in the Laplace rule:

$$\xi_{\text{MDP}}^{KT}(o_t \mid h_{<t}) = \frac{n_{a_{t-1}o_{t-1} \cdot o_t}^t + 0.5}{n_{a_{t-1}o_{t-1}}^t + 1}. \text{ Both are simple, regularized frequency estimators.}$$

Python: Laplace vs. KT Estimator, By Hand

Suppose the pair (action = 1, observation = 1) has occurred $n = 5$ times in the history so far, and the observation that followed it was, in order, 1, 0, 1, 1, 0 (so 3 of the 5 were followed by a 1).

Running estimates after each new data point:

```
1 followups = [1, 0, 1, 1, 0]          # o' observed after each occurrence of pair (a=1,o=1)
2 n, n1 = 0, 0
3 print(f"{'n':>2} {'n1':>3} {'Laplace (n1+1)/(n+2)':>22} {'KT (n1+0.5)/(n+1)':>18}")
4 for outcome in followups:
5     n += 1
6     n1 += outcome
7     laplace = (n1 + 1) / (n + 2)
8     kt      = (n1 + 0.5) / (n + 1)
9     print(f"{n:>2} {n1:>3} {laplace:>22.4f} {kt:>18.4f}")
```

n	n1	Laplace (n1+1)/(n+2)	KT (n1+0.5)/(n+1)
1	1	0.6667	0.7500
2	1	0.5000	0.5000
3	2	0.6000	0.6250
4	3	0.6667	0.7000
5	3	0.5714	0.5833

Both estimators track the empirical frequency n_1/n closely but pull it slightly toward 0.5, especially when n is small (compare row $n = 1$: raw frequency would say 1.0, but Laplace says only 0.667 and KT 0.75). This is exactly the regularizing effect of the Bayesian prior.

Algorithm 11.1: Computing $Q_{\xi_{\text{MDP}}}^{*,m}$ I

We now have everything needed to compute $Q_{\xi_{\text{MDP}}}^{*,m}(h_{<t}, a_t)$ via (11.2.1). Algorithm 11.1 spells out this recursion explicitly for the special case of binary action and observation spaces relevant to 2×2 matrix games (the general case, for any $|\mathcal{A} \times \mathcal{O}|$, is essentially the same).

Algorithm 11.1 (Q-value function of AIXI-MDP $Q_{\xi_{\text{MDP}}}^{*,m}(h_{<t}, a_t)$)

Require: horizon m ; MDP estimator ξ_{MDP} (Eq. 11.2.2); reward matrix $\mathfrak{R} \in \mathbb{R}^{2 \times 2}$.

Input: history $h_{<t}$, action a_t .

Output: Q-value $Q_{\xi_{\text{MDP}}}^{*,m}(h_{<t}, a_t)$.

- 1 **if** $m < t$ **then return** 0
- 2 **return** $\xi_{\text{MDP}}(0 \mid h_{<t}) \cdot [\mathfrak{R}(a_t, 0) + \max\{Q_{\xi_{\text{MDP}}}^{*,m}(h_{<t}a_t0, 0), Q_{\xi_{\text{MDP}}}^{*,m}(h_{<t}a_t0, 1)\}]$
 $+ \xi_{\text{MDP}}(1 \mid h_{<t}) \cdot [\mathfrak{R}(a_t, 1) + \max\{Q_{\xi_{\text{MDP}}}^{*,m}(h_{<t}a_t1, 0), Q_{\xi_{\text{MDP}}}^{*,m}(h_{<t}a_t1, 1)\}]$

Line 1 is the base case: once the current time t has run past the horizon m , there is no more value to be had, so $Q = 0$.

Line 2 is the recursive case: it exactly implements the sum in (11.2.1) for the two possible next observations $o_t \in \{0, 1\}$,

Algorithm 11.1: Computing $Q_{\xi_{\text{MDP}}}^{*,m}$ II

and for each one, plugs in the best of the two possible next actions (max over $a_{t+1} \in \{0, 1\}$) by calling the algorithm again, one step deeper, on the extended history $h_{<t}a_t o_t$.

Algorithm 11.1: Computing $Q_{\xi_{\text{MDP}}}^{*,m}$ III

Cost of the recursion. Each call to the algorithm recursively calls itself 4 times (twice for the two values of o_t , and within each, the max needs both possible a_{t+1} values). In general, this factor is $|\mathcal{A} \times \mathcal{O}| = 4$ times per level. So the total running time is $O(|\mathcal{A} \times \mathcal{O}|^{m-t+1}) = O(4^{m-t+1})$. This would take “forever” if we set m to the true lifetime of a long-lived agent, so in practice we impose a much shorter **moving horizon**

$$m_t = t + d - 1 \quad (\text{Definition 6.4.2})$$

looking only d time steps ahead, giving run time $O(4^d)$, feasible for small d .

A moving horizon means: at every time step t , re-plan as if the world will end d steps from now, take the resulting best immediate action, then move on to $t+1$ and repeat with a horizon that has itself moved one step further into the future. This keeps the planning problem a constant size (d steps), no matter how long the agent has already been running.

Algorithm 11.1: Computing $Q_{\xi_{\text{MDP}}}^{*,m}$ IV

A subtlety: time-inconsistency. A pure moving horizon can lead to *time-inconsistent* behaviour (Section 6.5) – for example, an agent that forever delays gratification, because there is always “one more step” of apparent future to exploit. To avoid this, in the experiments below, AIXI-MDP instead uses a **constant horizon** scheme: it fixes $m_t = d_{\max}$ and lets the effective lookahead $d_t := m - t + 1$ count down from $d_{\max}$ to 2, then *resets* m_t back up to $t + d_{\max}$, and repeats this cycle. In the experiments, the maximal lookahead is set to $d_{\max} = 8$.

The AIXI-MDP Agent

Putting it all together, the optimal action of AIXI-MDP at time t is

The AIXI-MDP action rule

$$a_t^{\text{MDP}} := \arg \max_{a_t} Q_{\xi_{\text{MDP}}}^{*, m_t}(h_{<t}, a_t)$$

At every time step, the agent calls Algorithm 11.1 twice – once for $a_t = 0$ and once for $a_t = 1$ – and picks whichever action has the higher computed Q -value. Ties can be broken arbitrarily.

For $t = 1, 2, 3, \dots$, AIXI-MDP and the opponent take **simultaneous** actions $(a_t, a'_t) := (a_t^{\text{MDP}}, o_t \sim \mu(o_t \mid h_{<t}))$. In theory the true environment μ should itself lie in $\mathcal{M}_{\text{MDP}}$ for the theory's guarantees to apply – but nothing stops us from testing AIXI-MDP against opponents that are *not* Markov, to see what happens (this is exactly what Section 11.3 does). In the experiments below, the maximal horizon is $d_{\text{max}} = 8$.

Python: Implementing Algorithm 11.1 I

A direct, small-scale implementation of the recursion above. To keep it simple we track the whole (hypothetical) history and count matching transitions directly from it, exactly following the definitions of n_{ao}^t and $n_{ao.o'}^t$:

```
1 R = {(0, 0): 3, (0, 1): 0, (1, 0): 5, (1, 1): 1}    # Prisoner's Dilemma payoffs
2
3 def xi_mdp(o, history, a_prev, o_prev):
4     """Laplace estimator (11.2.2): P(next obs = o | last pair (a_prev,o_prev))."""
5     if a_prev is None:                               # t = 1: no data yet -> uniform prior
6         return 0.5
7     n, n1 = 0, 0
8     for i in range(1, len(history)):                 # scan consecutive pairs in h_{<t}
9         a_tau, o_tau = history[i - 1]
10        o_next = history[i][1]
11        if (a_tau, o_tau) == (a_prev, o_prev):
12            n += 1
13            n1 += (o_next == 1)
14        p1 = (n1 + 1) / (n + 2)
15        return p1 if o == 1 else 1 - p1
16
17 def Q_star(history, a_t, m):
18     """Algorithm 11.1:  $Q^{*,m}_{-xi}(h_{<t}, a_t)$ , history =  $h_{<t}$ ."""
19     t = len(history) + 1
```

Python: Implementing Algorithm 11.1 II

```
20 if m < t:
21     return 0.0
22 a_prev, o_prev = history[-1] if history else (None, None)
23 total = 0.0
24 for o_t in (0, 1):
25     p = xi_mdp(o_t, history, a_prev, o_prev)
26     best_next = max(Q_star(history + [(a_t, o_t)], a_next, m)
27                    for a_next in (0, 1))
28     total += p * (R[(a_t, o_t)] + best_next)
29 return total
```

Python: Implementing Algorithm 11.1 III

Running this against a **1-tit-for-tat** opponent (cooperates first, then mimics the agent's previous action) with a small lookahead $d = 3$:

```
1 history, d = [], 3
2 for t in range(1, 11):
3     m = t + d - 1                                # moving horizon
4     qs = {a: Q_star(history, a, m) for a in (0, 1)}
5     a_t = max(qs, key=qs.get)                    # arg max_a Q*
6     a_opp = 0 if t == 1 else history[-1][0]      # 1-tit-for-tat
7     print(f"t={t:2d}   a_t={a_t}   o_t={a_opp}   Q(0)={qs[0]:.2f}   Q(1)={qs[1]:.2f}")
8     history.append((a_t, a_opp))
```

```
t= 1  a_t=1  o_t=0  Q(0)=7.50  Q(1)=9.00
t= 2  a_t=1  o_t=1  Q(0)=7.50  Q(1)=9.33
t= 3  a_t=1  o_t=1  Q(0)=7.17  Q(1)=8.11
t= 4  a_t=1  o_t=1  Q(0)=6.33  Q(1)=6.73
t= 5  a_t=1  o_t=1  Q(0)=5.92  Q(1)=5.97
t= 6  a_t=0  o_t=1  Q(0)=5.67  Q(1)=5.58
```

Python: Implementing Algorithm 11.1 IV

With only a 3-step lookahead the agent mostly still defects (matching the myopic, backward-induction intuition of finite-horizon Prisoner's Dilemma), though by round 6 cooperation has become marginally more attractive as the Q-values converge. Section 11.3 shows that with the book's full lookahead of $d_{\max} = 8$ over 100 rounds, AIXI-MDP reliably learns to cooperate against 1-tit-for-tat (Figure 11.2a) – longer lookahead lets the value of sustained mutual cooperation outweigh the one-off temptation to defect.

Setup of the Experiments

AIXI-MDP's performance was measured on the five classic 2×2 matrix games introduced in Section 10.2.3:

- **Prisoner's Dilemma** – mutual cooperation beats mutual defection, but each player is individually tempted to defect.
- **Stag Hunt** – similar to PD, but if both cooperate they earn even more than if they both defect.
- **Chicken** – both “going straight” is disastrous for both; the good outcomes require one player to “swerve.”
- **Battle of the Sexes** – both players want to coordinate on the *same* action, but disagree on which action is best.
- **Matching Pennies** – the only **zero-sum** game here: one player's gain is exactly the other's loss.

The opponent AIXI-MDP faced differed from game to game, since each game has different known “sensible” strategies to test against. AIXI-MDP was also tested against *itself* with different horizons in some games, and against the *Follow or Explore* (FoE) agent [PH06b] (which we will not elaborate on here) – it generally beats FoE.

All Five Games at a Glance

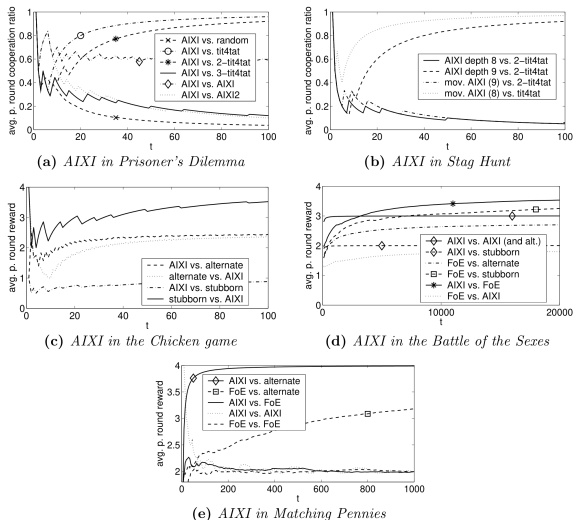
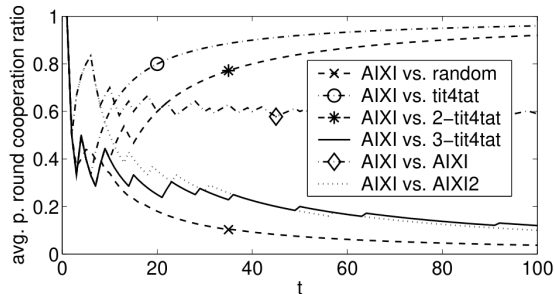


Figure 11.2: Performance of AIXI-MDP on the classical 2x2 matrix games against fixed strategies and against a version of itself [PH06b].

Prisoner's Dilemma



(a) *AIXI in Prisoner's Dilemma*

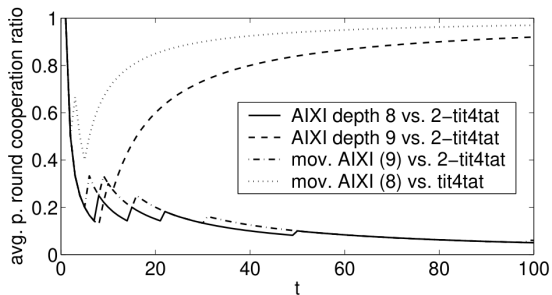
AIXI-MDP was compared to: *random* (a coin flip each round), and *n-tit-for-tat* for $n = 1, 2, 3$ (cooperates first, then cooperates only if the opponent cooperated the last n rounds in a row).

Learns to cooperate against *1-tit-for-tat* and *2-tit-for-tat*.

Learns to defect against *random* and *3-tit-for-tat*.

Why the split? *1-tit-for-tat* is Markov (its next action depends only on the agent's single last action), so it genuinely lies in AIXI-MDP's model class $\mathcal{M}_{\text{MDP}}$, and the self-optimization guarantee (Theorem 7.3.6) can kick in. *n-tit-for-tat* for $n \geq 2$ is **not** Markov, yet AIXI-MDP still learns to cooperate against *2-tit-for-tat*, though not against the harder *3-tit-for-tat* – it isn't given enough exploration to discover that deeper pattern. Against *random*, defection is never punished, so always defecting really is reward-maximizing. Against a copy of itself, symmetry makes both agents pick the same action every round; breaking that symmetry (e.g. different horizons) instead made both agents believe the other would defect, spiraling into mutual defection.

Stag Hunt



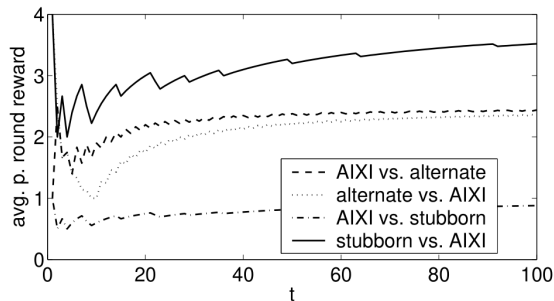
(b) *AIXI in Stag Hunt*

Stag Hunt resembles Prisoner's Dilemma, but mutual cooperation pays even *more* than mutual defection. AIXI-MDP was tested against *2-tit-for-tat*.

A **horizon-8** AIXI-MDP failed to learn to cooperate; a **horizon-9** AIXI-MDP succeeded (but still failed against the harder *3-tit-for-tat*).

One extra step of lookahead was enough to tip the balance from “defect” to “cooperate” here. Although an environment containing a 2-tit-for-tat opponent is not itself in the MDP model class $\mathcal{M}_{\text{MDP}}$, the mixture ξ_{MDP} can still converge to behave like the true environment μ even when $\mu \notin \mathcal{M}_{\text{MDP}}$ (Corollary 3.4.2) – model misspecification does not automatically doom the agent.

Chicken

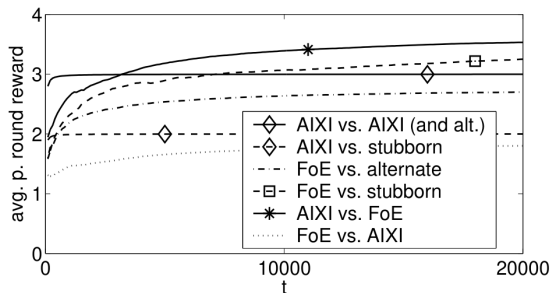


(c) *AIXI in the Chicken game*

In Chicken, the best strategy is to *alternate* between going straight and swerving, letting the opponent swerve most of the time. Opponents tested: *alternating* (alternates every round) and *stubborn* (only swerves after the agent has gone straight **3** times in a row).

AIXI-MDP quickly learns to go straight at the right moments against alternating. Against stubborn, however, it struggles to go straight three rounds in a row and so never discovers the optimal policy – yet against a 2-stubborn variant (only requiring two straight-goes in a row) it does learn the optimal sequence. This shows a real limitation: AIXI-MDP can fail to learn policies that require sustaining an unusual action for long enough to trigger a delayed payoff, when its lookahead is shorter than the required streak. When playing against itself, whichever copy has the higher horizon learns to go straight more often, and ends up earning more reward overall.

Battle of the Sexes

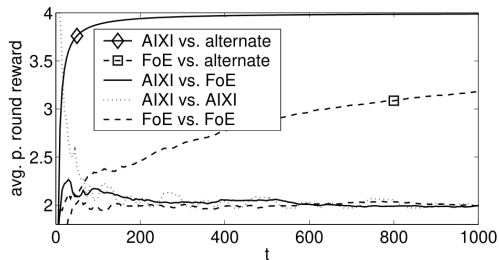


(d) AIXI in the Battle of the Sexes

Against *alternating*, AIXI-MDP learns to mirror the same alternation and achieves the socially optimal reward. Against *stubborn*, it fails to learn the required alternating sequence and earns a below-average reward each round. When two AIXI-MDP copies play each other, **both** learn to alternate: each agent believes its opponent is Markov (depends only on the very last round), which makes “alternate” look like the right response to an opponent who alternates – a mutually self-fulfilling prophecy, rather than deliberate coordination. Even so, both agents end up sharing the socially optimal average reward of 3 each round.

Both players want to take the *same* action each round, and, to maximize joint reward, should also *alternate* which action they coordinate on (so both players get a turn receiving the larger payoff). Opponents tested: *alternating* and *stubborn*.

Matching Pennies



(e) *AIXI in Matching Pennies*

The only **zero-sum** game tested. The game-theoretic minimax strategy is to play each action with equal (50/50) probability.

Against alternating, AIXI-MDP learns the pattern and exploits it fully, achieving the maximum possible reward of 4 every round – far better than the cautious 50/50 minimax strategy would give it, since the opponent here is exploitable. When two AIXI-MDP copies play each other with the same horizon, the perfect symmetry of a zero-sum game locks them into always choosing the same (matching) action; as soon as that symmetry is broken (e.g. different horizons), both agents learn to alternate between their two actions instead.

Exercises

Exercise numbers such as [C25i] are difficulty ratings (roughly, estimated hours of work; higher is harder), and a leading C marks an exercise that requires actual coding/implementation rather than pen-and-paper derivation.

- 1 [C25i] Implement AIXI-MDP in your chosen language and test it against the agents described in the experiments section (Section 11.3).
- 2 [C20] Generalize AIXI-MDP to non-binary actions and observations, and test it on more complicated environments such as gridworlds.
- 3 [C20] Determine the optimal agents for the given environment and opponents: tit-for-tat, stubborn, random, and alternating, etc.
- 4 [C23] Given that AIXI-MDP assumes the environment is Markov, find a non-Markov agent which you expect AIXI-MDP to perform well against, and test it empirically.

History and References

- AIXI-MDP was first introduced in [PH06b], where it was compared against **Acting with Expert Advice** [PH05b], also called **Follow or Explore (FoE)** – the *active-learning* version of *Prediction with Expert Advice* [HP04, HP05], which requires exploration. This extra exploration is achieved by forcing the FoE agent to explore with some exploration rate at every time step, a rate that decreases over time.
- The Bayesian approach (such as AIXI-MDP), expert-advice methods, and MDL-based (*minimum description length*) methods were compared in the online-learning setting in [Pol06].
- The relationship between *truncated* and *untruncated discounted* value functions – relevant to the moving-horizon and time-inconsistency issues discussed above – has been explored in [Hut06a, ASDH24].

Summary

- AIXI-MDP is the first, simplest, and most tractable computable approximation of AIXI: restrict the model class to Markov Decision Processes, and the Bayes-optimal policy becomes brute-force computable via Algorithm 11.1.
- The Laplace estimator ξ_{MDP} is a cheap, incremental, provably sound way to estimate MDP transition probabilities.
- The price of tractability is a restrictive assumption (Markov, no Grain of Truth): AIXI-MDP reliably learns against simple, Markov-compatible opponents, but can fail against opponents whose strategies require longer memory or longer sustained commitment than its lookahead allows.